

Programmation Dynamique

Capes option Info



Irénée Briquel
irenee.briquel@cyu.fr - Bureau A467

CY Cergy Paris Université

2022

Techniques de programmation usuelles

- Diviser pour régner
- Algorithmes gloutons
- Programmation dynamique

Plan du cours

Programmation dynamique

Compléments : mémoïsation

Compléments : backtracking

Champ d'application

On part d'un problème avec une définition récursive, mais plusieurs appels récursifs.

Exemple : Fibonacci :

$$\text{fibonacci}(n) = \begin{cases} 1, & \text{si } n = 0 \text{ ou } n = 1; \\ \text{fibonacci}(n-1) + \text{fibonacci}(n-2) & \text{sinon.} \end{cases}$$

Idée de la programmation dynamique

- Identifier les sous-problèmes qui seront appelés.
Ex : `fibonacci(n)` appellera `fibonacci(k)` pour $k \leq n$.
- Calculer ces sous-problèmes dans l'ordre croissant, stocker les résultats dans un tableau;
- Consulter le tableau lorsqu'on a besoin des solutions du sous-problème.

Fibonacci en programmation dynamique

```
def fibo(n):  
    #tableau pour stocker les valeurs de la suite :  
    val_fibo = [0]*(n+1)  
    val_fibo[0]=1  
    val_fibo[1]=1  
  
    #calcul des valeurs de la suite dans l'ordre  
  
    for i in range(2,n+1):  
        val_fibo[i] = val_fibo[i-1]+val_fibo[i-2]  
  
    return val_fibo[i]
```

Exemple2 : le rendu de monnaie

Rendu de monnaie

- **Entrée** : Une liste L de pièces existantes, une somme s à rendre.
- **Question** : Quel est le nombre minimal de pièces pour rendre s ?

Le rendu de monnaie (2)

- **Algorithme glouton** : : rendre un maximum de chaque pièce, de la plus grosse à la plus petite.
- Que se passe-t-il sur le système de pièces : $[1, 4, 5]$?

Le rendu de monnaie (2)

- **Algorithme glouton** : : rendre un maximum de chaque pièce, de la plus grosse à la plus petite.
- Que se passe-t-il sur le système de pièces : $[1, 4, 5]$?

Pour rendre 8, l'algorithme glouton n'a pas la solution optimale !

Le rendu de monnaie (2)

- **Algorithme glouton** : : rendre un maximum de chaque pièce, de la plus grosse à la plus petite.
- Que se passe-t-il sur le système de pièces : $[1, 4, 5]$?
Pour rendre 8, l'algorithme glouton n'a pas la solution optimale !
- Une définition par récurrence ?

Le rendu de monnaie (2)

- **Algorithme glouton** : : rendre un maximum de chaque pièce, de la plus grosse à la plus petite.
- Que se passe-t-il sur le système de pièces : $[1, 4, 5]$?

Pour rendre 8, l'algorithme glouton n'a pas la solution optimale !

- Une définition par récurrence ?

$$\text{rendu}(n) = \min_{p \in L \wedge p \leq n} (\text{rendu}(n - p) + 1)$$

En deux dimensions

Distance de Levenshtein (a,b)

Nombre minimal d'insertions, de suppressions, de substitutions pour passer de la chaîne de caractères a à la chaîne b.

Exemple : `lev("chien", "niche")` ?

Distance de Levenshtein : définition récursive

- On note $a[: -1]$ la chaîne a sans son dernier caractère.

Distance de Levenshtein : définition récursive

- On note $a[: -1]$ la chaîne a sans son dernier caractère.

$$\text{lev}(a, b) = \begin{cases} |a| & \text{si } |b| = 0, \\ |b| & \text{si } |a| = 0, \\ \text{lev}(a[: -1], b[: -1]) & \text{si } a[-1] = b[-1], \\ 1 + \min \begin{cases} \text{lev}(a[: -1], b) \\ \text{lev}(a, b[: -1]) \\ \text{lev}(a[: -1], b[: -1]) \end{cases} & \text{sinon.} \end{cases}$$

Solution en programmation dynamique

- On pose

$$\text{dist}[i][j] := \text{lev}(a[0\dots i], b[0\dots j])$$

- On calcule les cases du tableau successivement.

Plan du cours

Programmation dynamique

Compléments : mémoïsation

Compléments : backtracking

Mémoïsation

- **Idée** : : on est sur un problème résoluble par programmation dynamique (ou récurrence).
- On ne veut pas s'embêter à identifier les sous-cas qui seront appelés.

Mémoïsation

- **Idée** : : on est sur un problème résoluble par programmation dynamique (ou récurrence).
- On ne veut pas s'embêter à identifier les sous-cas qui seront appelés.
- **Solution** : écrire une fonction récursive, qui commence par consulter un dictionnaire des entrées déjà rencontrées.

Plan du cours

Programmation dynamique

Compléments : mémoisation

Compléments : backtracking

Backtracking - retour sur trace.

Exemple : rendu de monnaie

- On souhaite non seulement le nombre de pièces minimum à rendre, mais aussi la composition.

Backtracking - retour sur trace.

Exemple : rendu de monnaie

- On souhaite non seulement le nombre de pièces minimum à rendre, mais aussi la composition.
- Idée : mémoriser, dans un autre tableau, les prédécesseurs de chaque valeur.

Backtracking - retour sur trace.

Exemple : rendu de monnaie

- On souhaite non seulement le nombre de pièces minimum à rendre, mais aussi la composition.
- Idée : mémoriser, dans un autre tableau, les prédécesseurs de chaque valeur.
- A à la fin : phase de retour sur trace pour retrouver le chemin optimal.